

Recruitment Task ~ scRNA-seq Analysis

Anjana S Kumar

2026-06-04

R Markdown

Section 1: Quality Assessment Before Processing

Q1.1 — Sequencing and Alignment Quality

The summary report shows high quality across all sequencing matrices. Starting from the total number of reads assigned to this library which is 66,601,887 with no short reads skipped. Validity of barcode and UMI is excellent at 97.4% and 99.9% respectively, indicating minimal issues with sequencing and library preparation. All the scores for Q30 are strong; 94.1% for barcodes, 92.7% for UMIs and 90.2% for RNA reads (Q30 RNA reads are expected to be lower than UMI and barcode and this value accounts to that). Alignment metrics are outstanding. 96.1% of reads mapped to the genome and 93.6% mapped confidently, indicating the reference genome is appropriate and library quality is high. Also, 80.8% of reads mapped confidently to the transcriptome which is distinctively above the recommended threshold of >30%. The report indicated that the introns are included and hence accounting for 31.3% of reads map to intronic regions. One score worth noting here is the antisense mapping rate which is slightly elevated at 7.9% but still under the concerning range of 10%. As of for cell detection, 1,232 cells were detected with a high fraction of reads in cells, confirming clean cell capture with minimal ambient RNA contamination. Median genes per cell (3,234) and median UMI counts (9,881) are robust, indicating good transcriptomic depth per cell.

Q1.2 — Knee Plot Interpretation

In the Knee Plot barcodes are plotted against UMI counts. Where each barcode corresponds to a GEM (droplet) that may or may not contain a cell. GEMs containing real cells are generally expected to have significantly more UMIs than empty GEMs. The plot has a typical “cliff and knee” shape - Cliff — the steep blue-to-grey transition where UMI counts drop sharply (straight line downwards). This marks the boundary between cell-associated barcodes and background. Knee — the grey region where empty GEM barcodes plateau at low UMI counts. cellranger calls the cells above the cliff (blue region) which accounts for real cell than background (grey region). In this sample the cliff occurs at approximately rank 1,000, yielding 1,232 estimated cells.

Q1.3 — Knee Plot Quality Assessment

This sample does represent a typical high-quality pattern when we compare it to the malformed knee plots. It does not have characteristics of either a compromised plot or a bi-modal heterogeneous plot. In this sample knee plot we can see a clear separation between the dark blue cell-associated barcodes (high values of UMIs) and the grey background barcodes (<500 UMIs).

Q1.4 — Addressing Potential Defects

If there were any defects in the pattern we could go for a solution according to the type of pattern : -No distinct cliff and Knee: This means the algorithm cannot distinguish between real intact cell and ambient RNA. In this case we can try emptyDrops from DropletUtils this can help with distinguishing the cell containing droplet from empty ones. -Double knee: May indicate cell multiplets or two distinct cell size populations we could apply DoubletFinder and consider filtering by UMI distribution. -If there is no proper separation because of high background: high background suggests significant ambient RNA contamination so we can try SoupX for ambient RNA correction before downstream analysis. -Very low cell UMIs overall: this may indicate poor quality of library so we can consider re-sequencing or applying more aggressive QC filtering. -Undercalling (showing cliff prematurely) - if this is the issue we can try changing some statistical parameters, adjusting the FDR to a lower value in DropletUtils so more cells can be included.

Q1.5 — Sequencing Depth Assessment

Yes, the sequencing depth is sufficient because, Sequencing Saturation is at 70.8% above the commonly recommended threshold of 50-60%. This means the majority of unique transcripts have already been captured through this sequencing. Additional sequencing would yield diminishing new information. According to 10x guidelines, a minimum of 20,000 reads per cell is recommended. This sample achieves 54,060 mean reads per cell, which is very high. Combining this with other aspects like median genes per cell (3,234) and total genes detected (24,694), the dataset is sufficiently deep for standard analyses.

Section 2: Data Processing

Q2.1 - Load Filtered Matrix and Evaluate QC

Metrics

Script

```
knitr::opts_chunk$set(echo = TRUE)
#library loading
library(Seurat)
library(ggplot2)
library(dplyr)
#Data loading
data_filtered <- Read10X(
  data.dir = "/Users/ask/Downloads/data_scRNA 2/filtered_feature_bc_matrix"
)
#create seurat object
seurat_obj <- CreateSeuratObject(
  counts = data_filtered,
  project = "JAC_demo",
  min.cells = 3,
  min.features = 200
)
#mitochondrial percentage
seurat_obj[["percent.mt"]] <- PercentageFeatureSet(
  seurat_obj,
  pattern = "^MT-"
)
```

###summary

```
cat("=== UMI Counts per Cell (nCount_RNA) ===\n")
```

```
## === UMI Counts per Cell (nCount_RNA) ===
```

```
summary(seurat_obj$nCount_RNA)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      500   5110   6832   7818   9213   56843
```

```
cat("\n=== Genes Detected per Cell (nFeature_RNA) ===\n")
```

```
##
## === Genes Detected per Cell (nFeature_RNA) ===
```

```
summary(seurat_obj$nFeature_RNA)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      215   1587   1936    2119   2534    6427
```

```
cat("\n=== Mitochondrial Content % (percent.mt) ===\n")
```

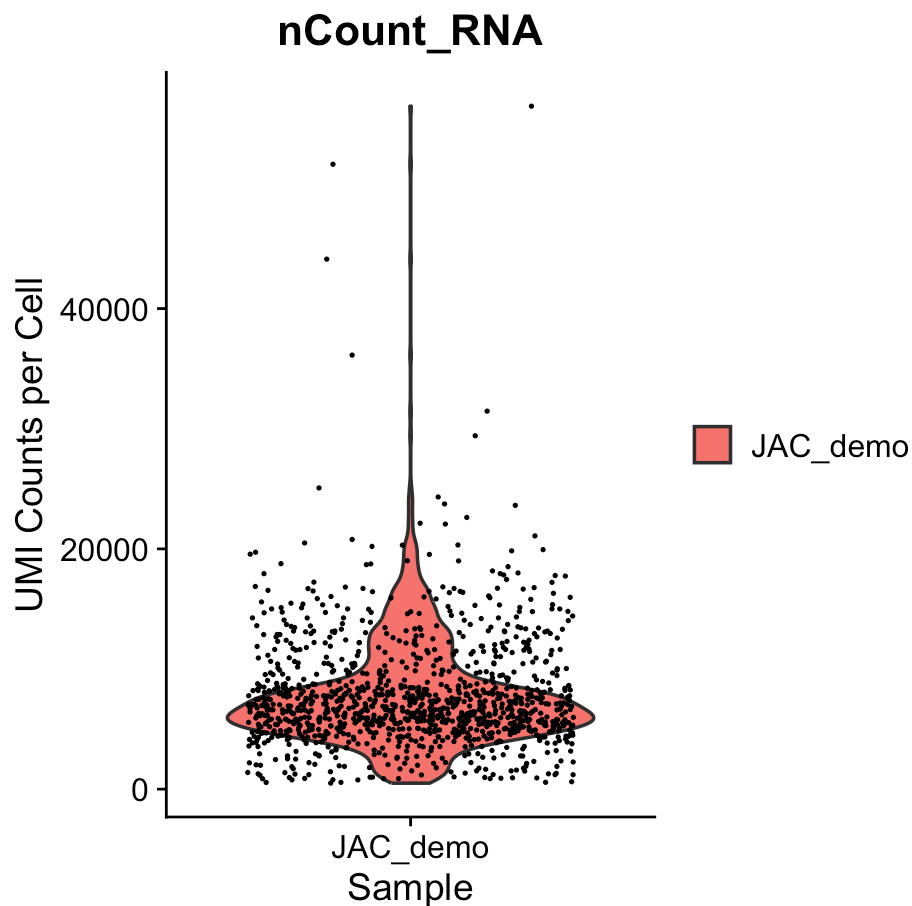
```
##
## === Mitochondrial Content % (percent.mt) ===
```

```
summary(seurat_obj$percent.mt)
```

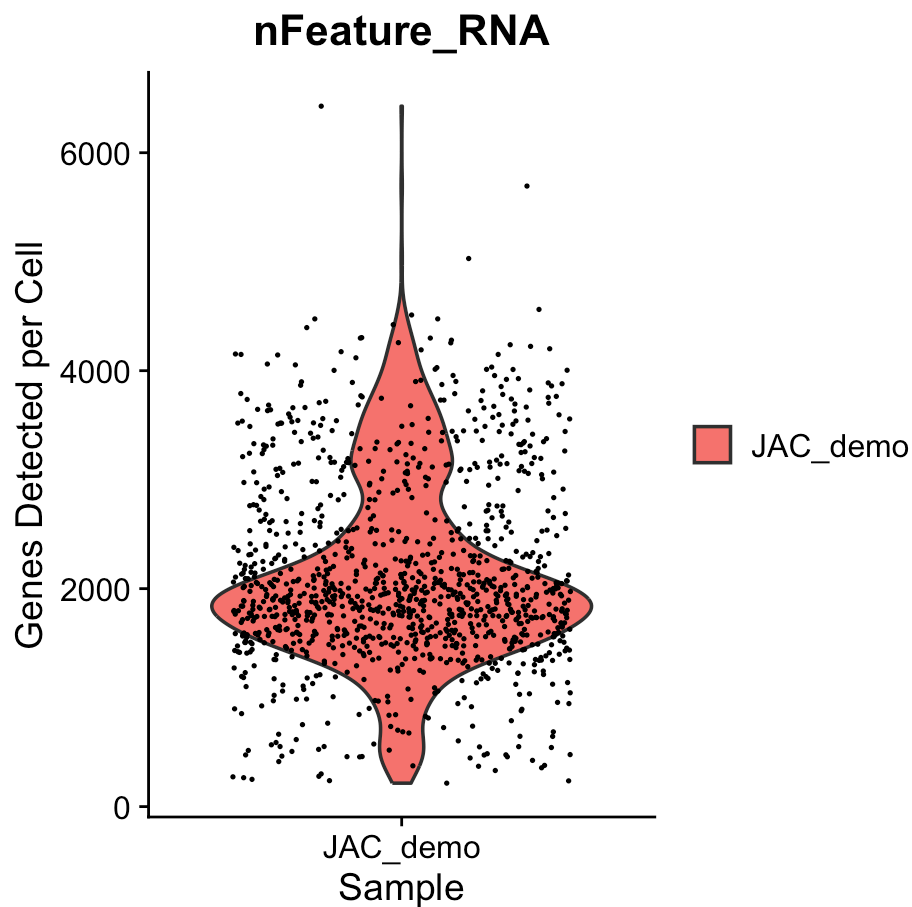
```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.04502  8.62118 10.14941 11.71017 12.18612 77.42885
```

```
###plots
```

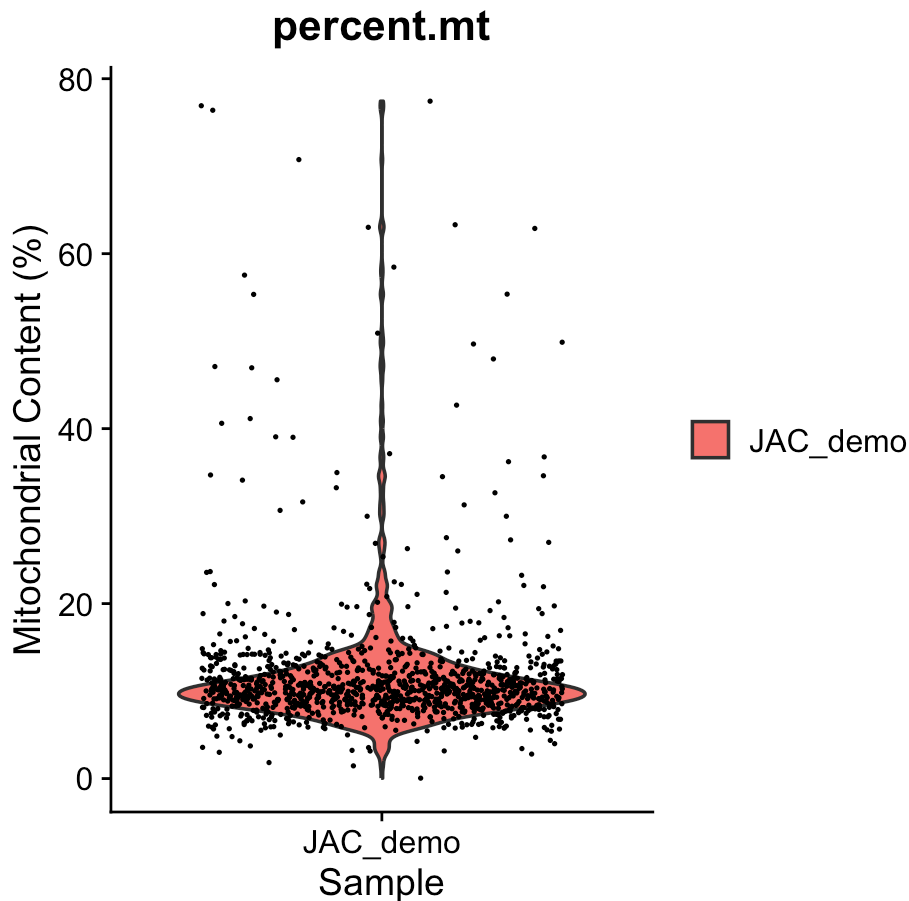
```
# UMI counts
VlnPlot(seurat_obj, features = "nCount_RNA", pt.size = 0.1) +
  theme(axis.text.x = element_text(angle = 0, hjust = 0.5)) +
  xlab("Sample") + ylab("UMI Counts per Cell")
```



```
# Genes detected
VlnPlot(seurat_obj, features = "nFeature_RNA", pt.size = 0.1) +
  theme(axis.text.x = element_text(angle = 0, hjust = 0.5)) +
  xlab("Sample") + ylab("Genes Detected per Cell")
```



```
# Mitochondrial content
VlnPlot(seurat_obj, features = "percent.mt", pt.size = 0.1) +
  theme(axis.text.x = element_text(angle = 0, hjust = 0.5)) +
  xlab("Sample") + ylab("Mitochondrial Content (%)")
```



###interpretation of plot and summary

- i. UMI distribution looks quite healthy as the median value falls at 6832 and an interquartile range of approximately 5,110–9,213 UMIs. The only noticeable issue seems to be the maximum value which is 56843, pretty high compared to min, mean and median value and expands the range. this could be due to outlier due to doublet or a very active cell.
- ii. Genes detected per cell with a median of 1,936 genes per cell reflects good transcriptomic complexity.while the distribution is relatively very tight around 1500-2500 for most of the cells, there could be a subset cells which does have genes <300 per cell since the min value (215) hits very close to threshold of 200.
- iii. Ideal Mitochondrial percentage for total cellular reads falls around 5% -15%.Here in this sample while the median falls at 10.14% (which is fine), the maximum peaks to 77% which is concerning as this is an extreme outlier.A cell with such mitochondrial content would be dead, dying or structurally compromised.

What we can do to try fixing the small issues

:-

For the UMI counts we can keep the lower threshold just as it is (500) but change the upper threshold to < 25000 to reduce the range and keep limited outliers or doublets. For the genes per cell lower and upper can be > 200 and < 5000 respectively as this would wipe out extreme outliers as our median is 1936.Two cells captured in one droplet would express approximately double the genes of a single cell, explaining the suspicious outliers reaching 6,427. And for the mitochondrial percentage we can keep upper limit as 20% to exclude the compromised cells.

Q2.2 Cell Debris detection with FeatureScatter and removal of cell debris

I have made 2 scatter plots to detect the cell debris :

- i. This scatter plot shows UMI counts against percentage of mitochondrial content in a cell (each dot indicates a cell). These features are negatively correlated to each other which is evident by the pearson correlation value of -0.29. And they are expected to be correlated negatively since a healthy cell is supposed to have lower amounts of mt content and moderate amounts of UMI counts. We can locate a healthy chunk of cells with low percent.mt and moderate UMI at lower left portion of the plot. Cell debris presents as a distinct population in the upper left — low UMI counts combined with very high mitochondrial percentage (>40%). These reflect compromised cells where cytoplasmic RNA has leaked out leaving majorly mitochondrial transcripts behind. We can also spot the debris cells which is reaching 77% (noted in Q 2.1).

(ii) The 2nd plot is UMI counts against number of genes per cell. This is a positive correlation as a healthy cell is supposed to have number of genes as well as UMI counts on the higher end together. This is complied by the pearson value we got (0.91) showing strong positive correlation. Here cell debris forms as a distinct population in the lower left of this plot. Which is low UMI counts combined with very few genes detected, deviating from the main correlation line. Likely doublets appear as outliers in the upper right, abnormally high UMIs and gene counts compared to the main population.

2- To remove cell debris : seurat's subset() function can be applied.

```
seurat_clean <- subset(
  seurat_obj,
  subset = nCount_RNA > 500 &
           nCount_RNA < 25000 &
           nFeature_RNA > 200 &
           nFeature_RNA < 5000 &
           percent.mt < 20
)

cat("Cells before filtering:", ncol(seurat_obj), "\n")
```

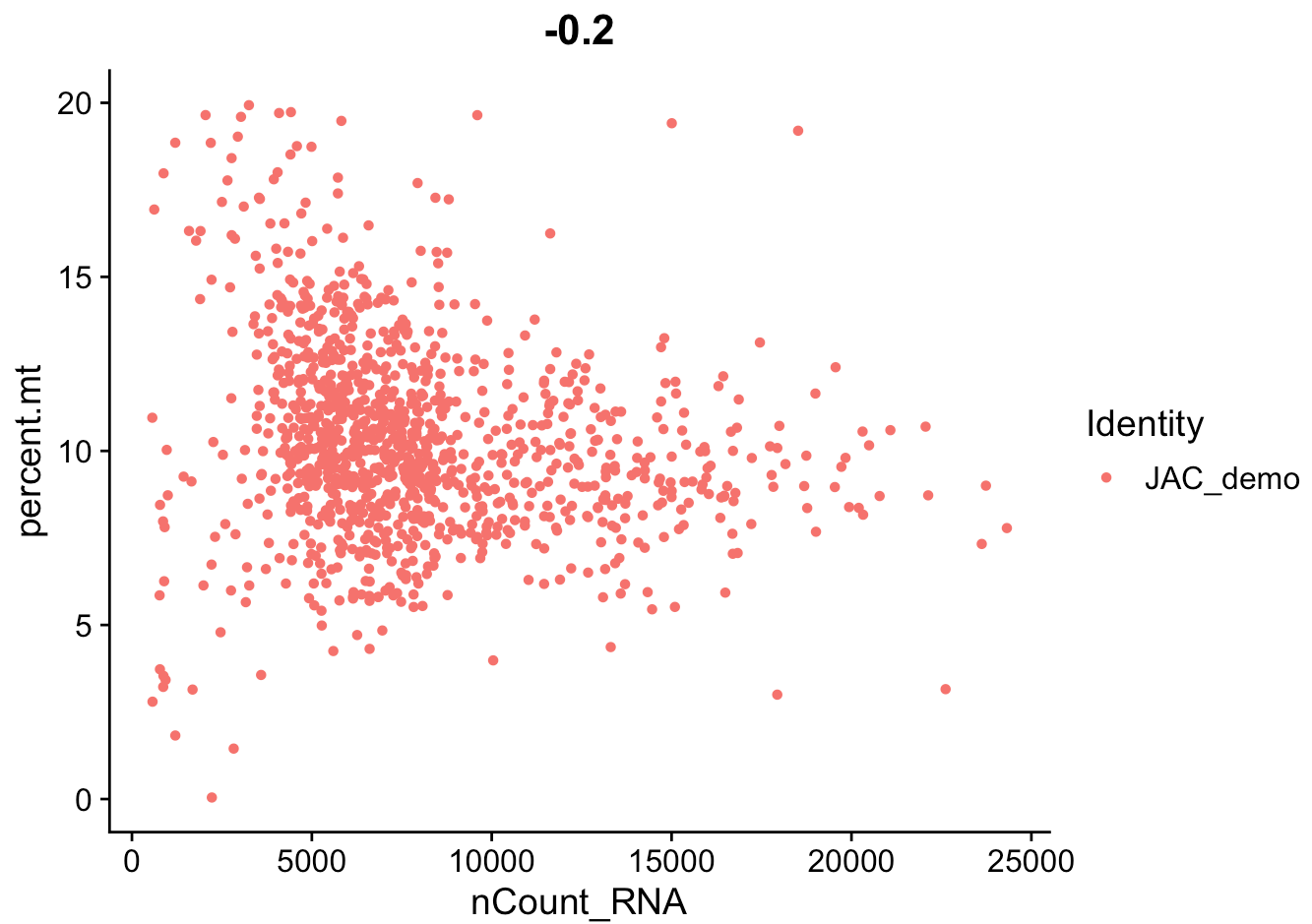
```
## Cells before filtering: 1176
```

```
cat("Cells after filtering:", ncol(seurat_clean), "\n")
```

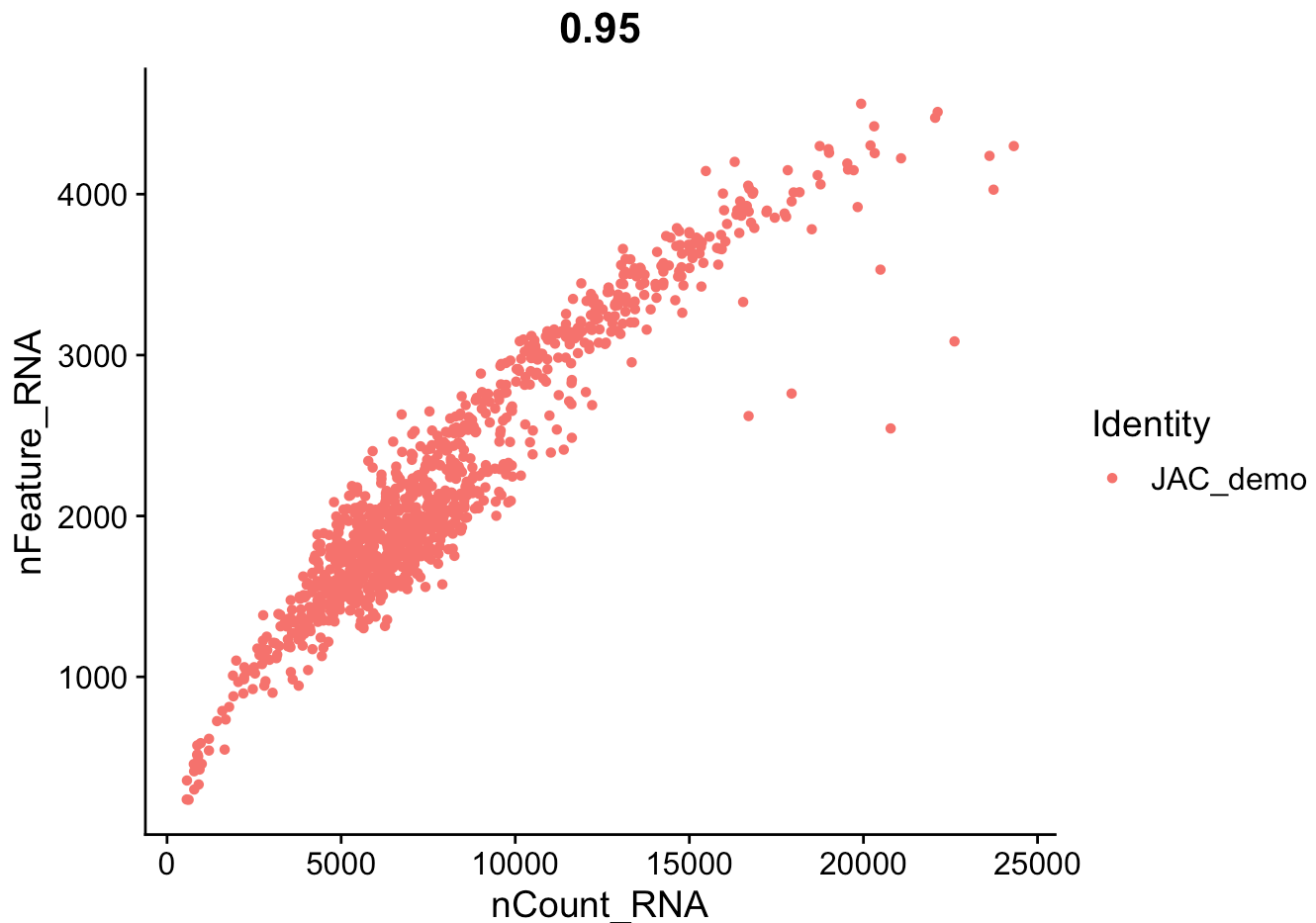
```
## Cells after filtering: 1105
```

Applying these thresholds to the sample reduced the dataset from 1,176 cells to 1,105 cells, removing 71 low-quality cells (6% of the dataset). ##new scatter plots with seurat_clean #All downstream analysis would be performed on seurat_clean (1105 cells)

```
FeatureScatter(seurat_clean,
               feature1 = "nCount_RNA",
               feature2 = "percent.mt")
```



```
FeatureScatter(seurat_clean,  
               feature1 = "nCount_RNA",  
               feature2 = "nFeature_RNA")
```

Post filtering scatter plots confirms the removal of high mitochondrial debris and extreme outliers.

Q2.3 Doublet Detection

While the basic threshold filtering (Q2.2) removes the obvious high UMI doublets, some doublets which fall in the normal range miss out through the basic threshold based filtering step hence dedicated algorithms are needed to detect them. ### Three Strategies for Doublet Detection

(i) DoubletFinder (Simulation based + Nearest Neighbors) Rationale: If a real cell is actually a doublet, its expression profile should resemble artificially created doublets. Therefore, real cells surrounded by many simulated doublets in PCA space are likely doublets. We can see the closest neighbors of a real cell and distinguish if it's a singlet or doublet. Question processed here: Does this cell look like the artificial doublets we generated? (Reference: McGinnis et al., Cell Systems, 2019)

(ii) DoubletDecon (Deconvolution) Rationale: This one uses a completely different idea behind the detection. Basic idea here is that a true doublet should be explainable as a mixture of two existing cell types. If a cell's expression can be decomposed into contributions from two cell populations, it is likely a doublet. Cells with unique expression signatures are rescued as real biological states. Question processed here: Can this cell be explained as a combination of two other cell types? (Reference: DePasquale et al., Cell Reports, 2019)

(iii) scds (scds is a package that offers two complementary approaches)

(a) cxds (Co-expression) Rationale: Different cell types often express mutually exclusive marker genes. If a cell expresses marker genes from two unrelated cell types simultaneously, it may be a doublet. Question processed here: Is this cell expressing gene combinations that shouldn't normally occur together?

(b)bcds (Binary Classification) Rationale: Generate artificial doublets and train a classifier to distinguish singlets from doublets. Cells classified as similar to artificial doublets receive high doublet scores. Question processed here: Does a machine-learning model think this cell looks like a simulated doublet?

(c)hybrid (combination of both) Rationale: Combines cxds and bcds scores to leverage both abnormal marker co-expression and classifier-based evidence. Question processed here: Do both biological evidence and machine learning suggest this is a doublet? (Reference: Bais & Kostka, Bioinformatics, 2020)

Application: cxds from scds

Note: For this sample's analysis, the hybrid method could not be run due to a version incompatibility between scds and the current xgboost installation. cxds was therefore applied independently.

```
knitr::opts_chunk$set(echo = TRUE)
# Load SingleCellExperiment and scds FIRST
library(SingleCellExperiment)
library(scds)
library(xgboost)

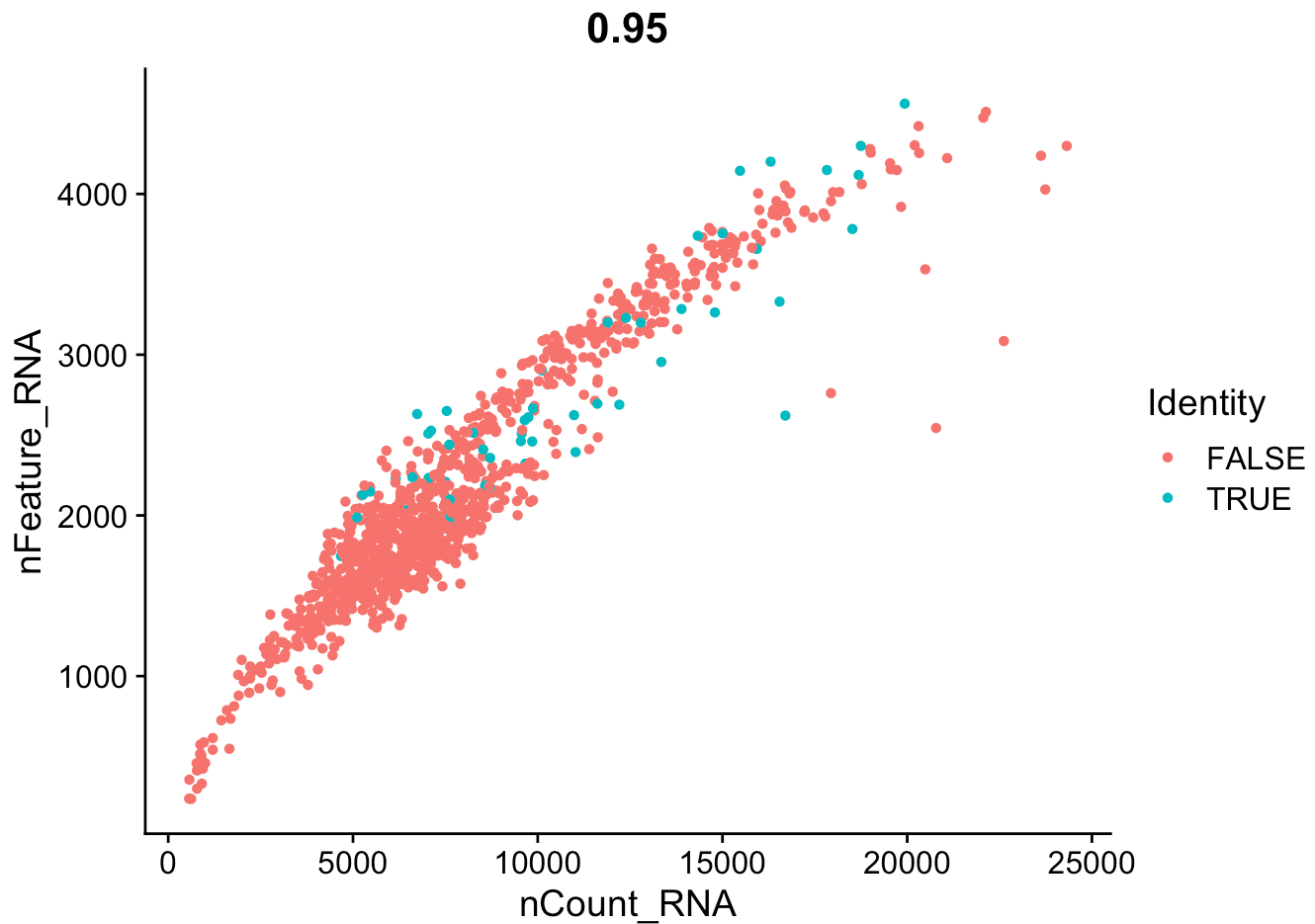
# Then Seurat and others
library(Seurat)
library(ggplot2)
library(dplyr)
sce <- as.SingleCellExperiment(seurat_clean) #convert seurat to singlecellexperiment
sce <- cxds(sce, retRes = TRUE)              #cxds only
summary(sce$cxds_score)
```

```
##      Min.   1st Qu.   Median     Mean   3rd Qu.     Max.
##    56.49  19737.71  30869.18  32836.04  43997.91 154458.79
```

```
doublet_threshold <- quantile(sce$cxds_score, 0.95) ### top 5% = doublets
sce$is_doublet <- sce$cxds_score > doublet_threshold
table(sce$is_doublet)
```

```
##
## FALSE  TRUE
##  1049    56
```

```
seurat_clean$cxds_score <- sce$cxds_score
seurat_clean$is_doublet <- sce$is_doublet ##scores added back to seurat object
##Scatter plot
FeatureScatter(seurat_clean,
               feature1 = "nCount_RNA",
               feature2 = "nFeature_RNA",
               group.by = "is_doublet")
```



```
#remove doublets
seurat_nodbl <- seurat_clean[, !seurat_clean$is_doublet] ## keep singlets only

cat("Cells before doublet removal:", ncol(seurat_clean), "\n")
```

```
## Cells before doublet removal: 1105
```

```
cat("Cells after doublet removal:", ncol(seurat_nodbl), "\n")
```

```
## Cells after doublet removal: 1049
```

```
cat("Doublets removed:", sum(seurat_clean$is_doublet), "\n")
```

```
## Doublets removed: 56
```

Interpretation

I applied `cxds` from `scds` package and it detected 56 doublets (5% of 1,105 cells). This is done by using a threshold at the 95th percentile of `cxds` scores. As theoretically expected these doublets (cells highlighted in blue) are mostly concentrated in the upper right corner with higher UMI counts and gene expression per cell. After removing 56 doublets, 1,049 high-confidence singlets remain for downstream analysis.

Q2.4 SoupX Ambient RNA Correction

Rationale behind SoupX : SoupX is used to estimate and correct cell-free ambient RNA contamination in droplet-based single-cell RNA sequencing data. Ambient RNA is an important confounding factor it can cause false positive expression of cell-type specific marker genes in unrelated cell types. For example, ADIPOQ from lysed adipocytes appearing falsely in neurons, reducing the biological accuracy and statistical significance of downstream results. SoupX first analyses empty droplets as a reference — these droplets contain no real cell and therefore represent pure ambient RNA (the “soup”). Using this soup profile, SoupX then estimates what fraction of each real cell’s counts comes from ambient RNA (the contamination fraction, ρ). Finally it subtracts the estimated soup contribution from each cell’s counts, returning corrected expression values representing only true cellular RNA.

Application to Demo Dataset

library(SoupX)

```
seurat_nodbl <- NormalizeData(seurat_nodbl)
seurat_nodbl <- FindVariableFeatures(seurat_nodbl)
seurat_nodbl <- ScaleData(seurat_nodbl)
seurat_nodbl <- RunPCA(seurat_nodbl)
seurat_nodbl <- FindNeighbors(seurat_nodbl, dims = 1:15)
seurat_nodbl <- FindClusters(seurat_nodbl, resolution = 0.5)
```

```
## Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck
##
## Number of nodes: 1049
## Number of edges: 32125
##
## Running Louvain algorithm...
## Maximum modularity in 10 random starts: 0.8868
## Number of communities: 8
## Elapsed time: 0 seconds
```

```

seurat_nodbl <- RunUMAP(seurat_nodbl, dims = 1:15)
# Loading both raw and filtered matrices for soupX to learn about ambient mRNA
raw <- Read10X("/Users/ask/Downloads/data_scRNA 2/raw_feature_bc_matrix")
filtered <- Read10X("/Users/ask/Downloads/data_scRNA 2/filtered_feature_bc_matrix")

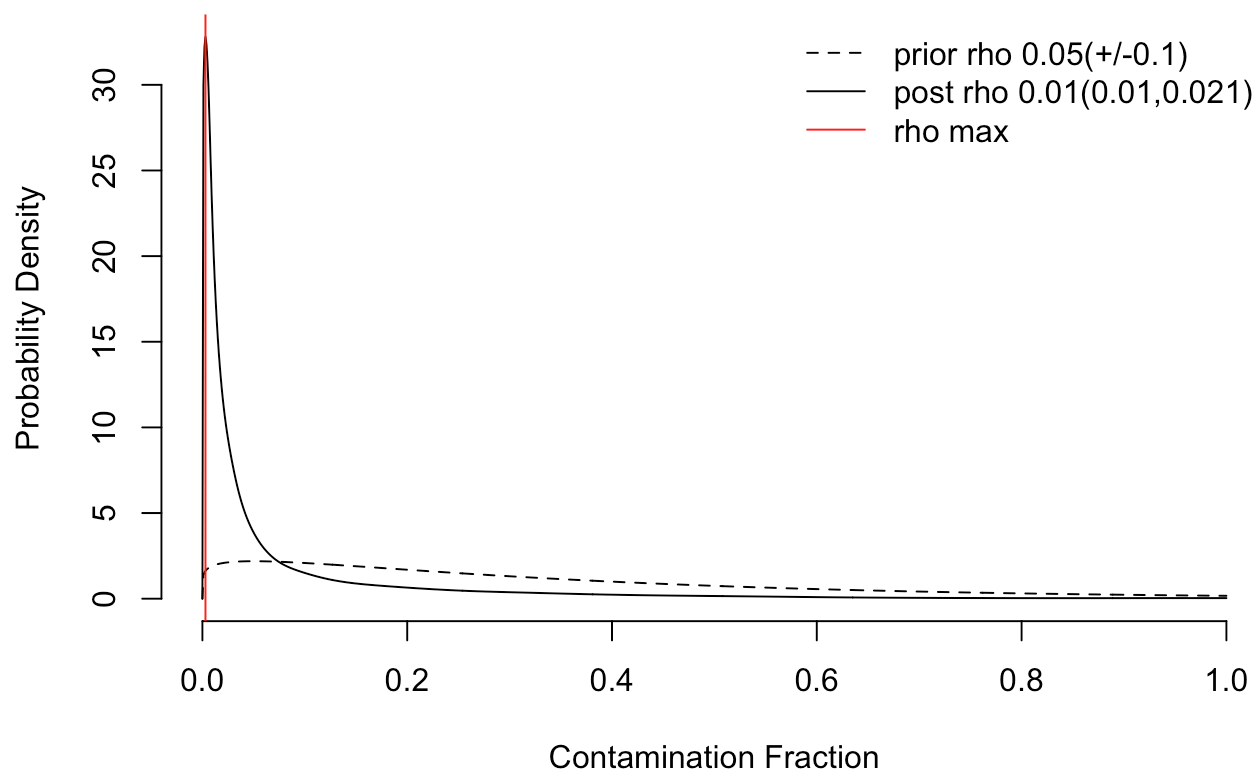
# Subsetting the filtered data to QC-passed cells (without the doublets) only so the clusters can form for 1049 cells
filtered_counts <- filtered[, colnames(filtered) %in% colnames(seurat_nodbl)]

# Creating SoupChannel object
sc <- SoupChannel(tod = raw, toc = filtered_counts)

# Adding cluster information (required for contamination estimation)
clusters_char <- as.character(seurat_nodbl$seurat_clusters)
names(clusters_char) <- colnames(seurat_nodbl)
sc <- setClusters(sc, clusters_char)

# Estimate contamination fraction automatically
sc <- autoEstCont(sc)

```



```
# Apply correction
out <- adjustCounts(sc)

# Add corrected matrix as new assay in Seurat object - seurat_nodbl
seurat_nodbl[["SoupXcounts"]] <- CreateAssayObject(counts = out)

# Verify if the Assay exists
Assays(seurat_nodbl)
```

```
## An object of class "SimpleAssays"
## Slot "data":
## List of length 1
```

```
cat("Contamination fraction (rho):", 0.01, "\n")
```

```
## Contamination fraction (rho): 0.01
```

Interpretation:

The estimated global contamination fraction (ρ) is 0.01 which is 1% indicating very low ambient RNA contamination in this dataset. Our prior assumption was 5% contamination (dashed line), but the data evidence strongly supports the lower value of 1% (solid line), with the red vertical line marking the maximum likelihood estimate. The corrected counts are stored in the SoupXcounts assay (default) alongside the original RNA assay in the Seurat object (seurat_nodbl).

Q2.5 Lognormalize VS SCTransform Single cell RNA seq data often gets confounded by technical factors including sequencing depth. Not all cells get sequenced in similar depth and hence we do not get UMI counts for all cells in uniformity. To resolve this we use normalisation techniques to give a common ground for all cells and we do not overestimate any specific cell and inflate statistical power or vice versa.

LogNormalize Rationale : Normalizes by total UMI counts then log transforms. **Method** : Divide by library size $\times 10,000$ then $\log_{10}$ transformation **Performance** : It is very straightforward method; Fast, memory efficient, is widely used and sufficient for most analyses where technical noise is not a major issue. **Limitation** : Does not model gene-specific noise, log transformation can distort statistics.

SCTransform Rationale : Models technical noise using regularized negative binomial regression **Method** : Fits GLM per gene using sequencing depth as covariate, returns Pearson residuals. For each gene it creates a statistical model. If a gene is expressed more than expected given the cell's sequencing depth it gives positive residual which is biologically meaningful. If a gene matches the expected technical noise it gives residual near zero which means it's technical. **Performance** : It is advanced and complex in nature, really good at finding rare cell types, Handles highly expressed genes better, Better for datasets with large sequencing depth variation. **Limitation** : Much slower than LogNormalize, More memory intensive, Can overcorrect in some datasets, Requires glmGamPoi package for speed.

Application

```
seurat_log <- NormalizeData(seurat_nodbl,  
                           normalization.method = "LogNormalize",  
                           scale.factor = 10000)  
seurat_log <- FindVariableFeatures(seurat_log)  
cat("LogNormalize done!\n")
```

```
## LogNormalize done!
```

```
seurat_sct <- SCTransform(seurat_nodbl,  
                          vars.to.regress = "percent.mt",  
                          verbose = FALSE)  
  
cat("SCTransform done!\n")
```

```
## SCTransform done!
```

##correlation

```
# Get normalized matrices  
log_matrix <- GetAssayData(seurat_log, assay = "RNA", layer = "data")  
sct_matrix <- GetAssayData(seurat_sct, assay = "SCT", layer = "data") #SCTransform cre  
ates a new SCT assay  
  
# Common genes  
common_genes <- intersect(rownames(log_matrix), rownames(sct_matrix))  
cat("Common genes:", length(common_genes), "\n")
```

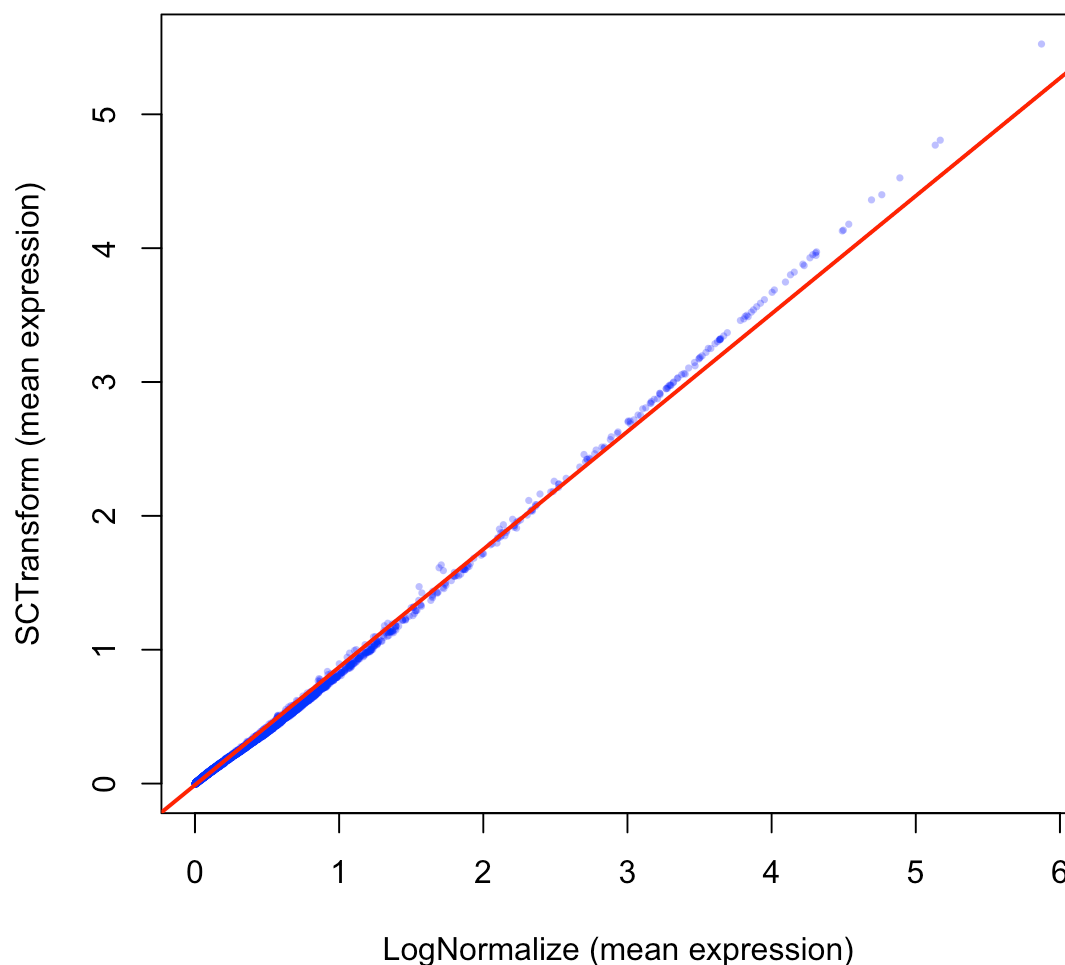
```
## Common genes: 13768
```

```
# Mean expression per gene  
log_mean <- rowMeans(log_matrix[common_genes, ])  
sct_mean <- rowMeans(sct_matrix[common_genes, ])  
  
# Correlation  
correlation <- cor(log_mean, sct_mean, method = "pearson")  
cat("Pearson correlation:", round(correlation, 3), "\n")
```

```
## Pearson correlation: 0.998
```

```
# Plot
plot(log_mean, sct_mean,
     xlab = "LogNormalize (mean expression)",
     ylab = "SCTransform (mean expression)",
     main = paste("Correlation between normalization methods\nr =",
                  round(correlation, 3)),
     pch = 16,
     col = rgb(0, 0, 1, 0.3),
     cex = 0.5)
abline(lm(sct_mean ~ log_mean), col = "red", lwd = 2)
```

Correlation between normalization methods $r = 0.998$



Interpretation

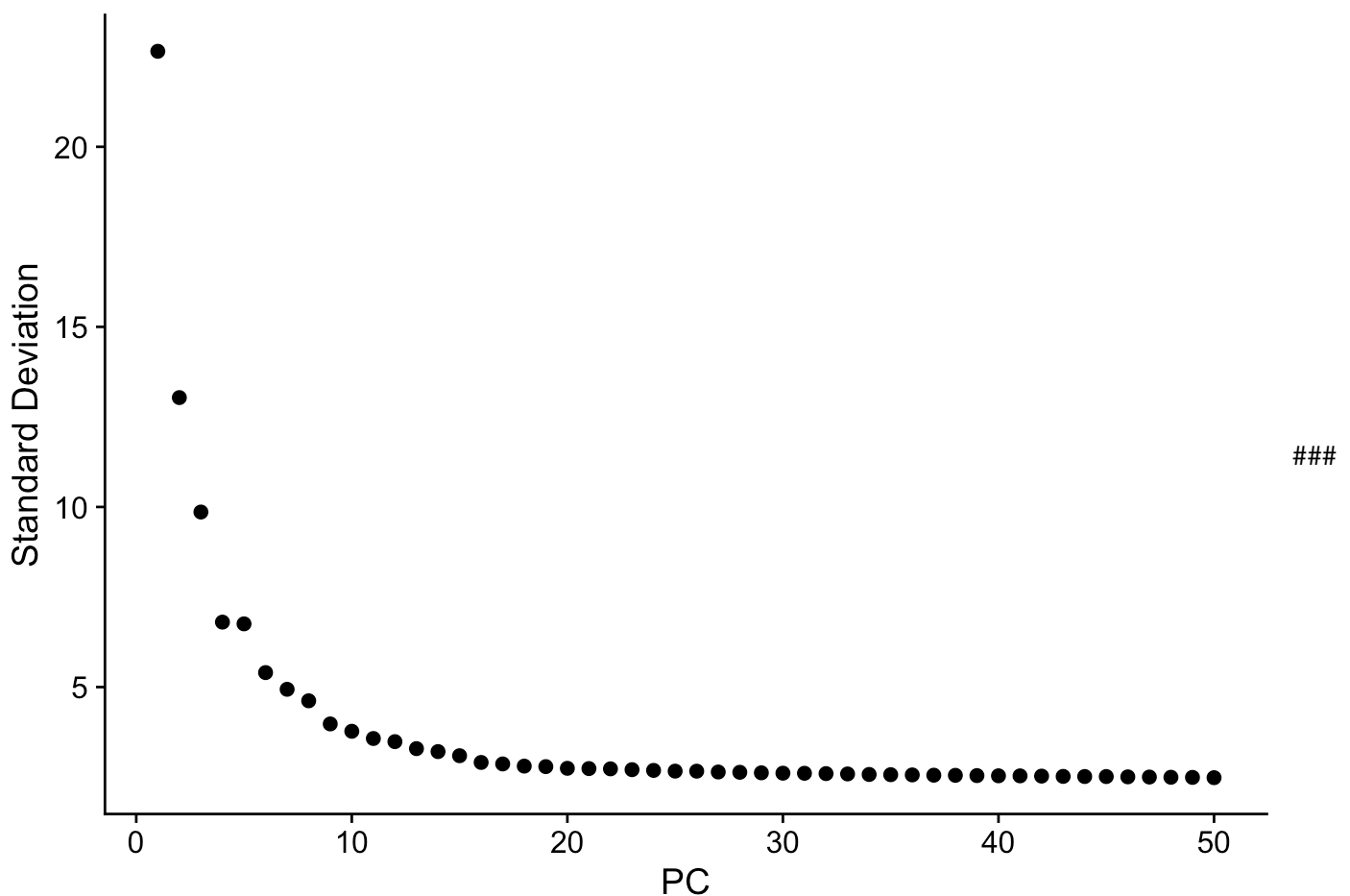
Pearson correlation between methods: 0.998 ; which is very strong. Both the methods show almost similar normalization results across 13,768 common genes. According to the Plot we can see the blue dots closely following the red line, there isn't much deviation other than slight departure close to the upper right corner. This is where the SCTransform is little more aggressive with normalisation on highly expressed genes.

Q2.6 — PCA and Optimal PC Selection

After normalization of dataset single cell data are supposed to be reduced dimensional. The first step usually for this is linear scaling which we can skip as SCTransform automatically does the scaling for the dataset. PCA (Principal component analysis) will help reduce the high dimensional gene expression space (13,768 genes) into a smaller number of principal components that capture the most meaningful biological variation. PCs progressively (1 to n) become less variant and just represent noise in later PCs hence it's important to choose a range of best PCs which captures real biological variance.

Application of PCA

```
DefaultAssay(seurat_sct) <- "SCT"  
seurat_sct <- RunPCA(seurat_sct, verbose = FALSE)  
ElbowPlot(seurat_sct, ndims = 50)
```



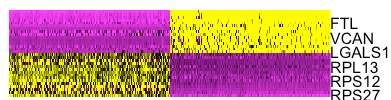
Strategies for PC Selection

Elbow Plot I am using elbow plot first to get an idea how many PCs to include - from what I can see is the first 5 PCs include maximum variance then there is a gradual decline and from PC15 the curve flattens, mostly noise.

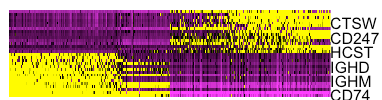
DimHeatmap (shows gene loadings per PC)

```
DimHeatmap(seurat_sct,
  dims = 1:15,
  cells = 500,
  balanced = TRUE)
```

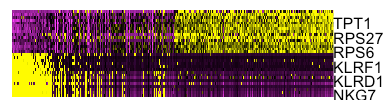
PC_1



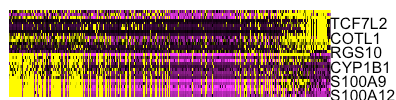
PC_2



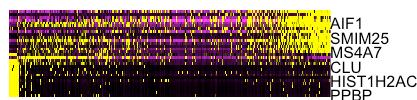
PC_3



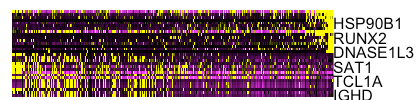
PC_4



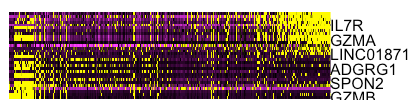
PC_5



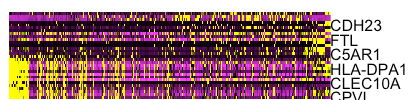
PC_6



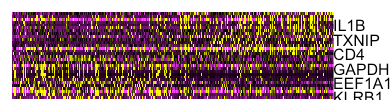
PC_7



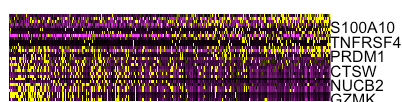
PC_8



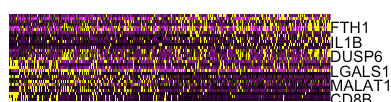
PC_9



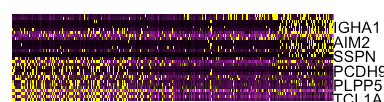
PC_10



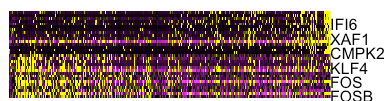
PC_11



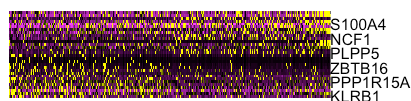
PC_12



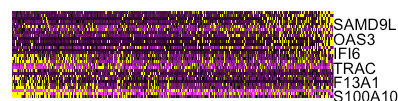
PC_13



PC_14

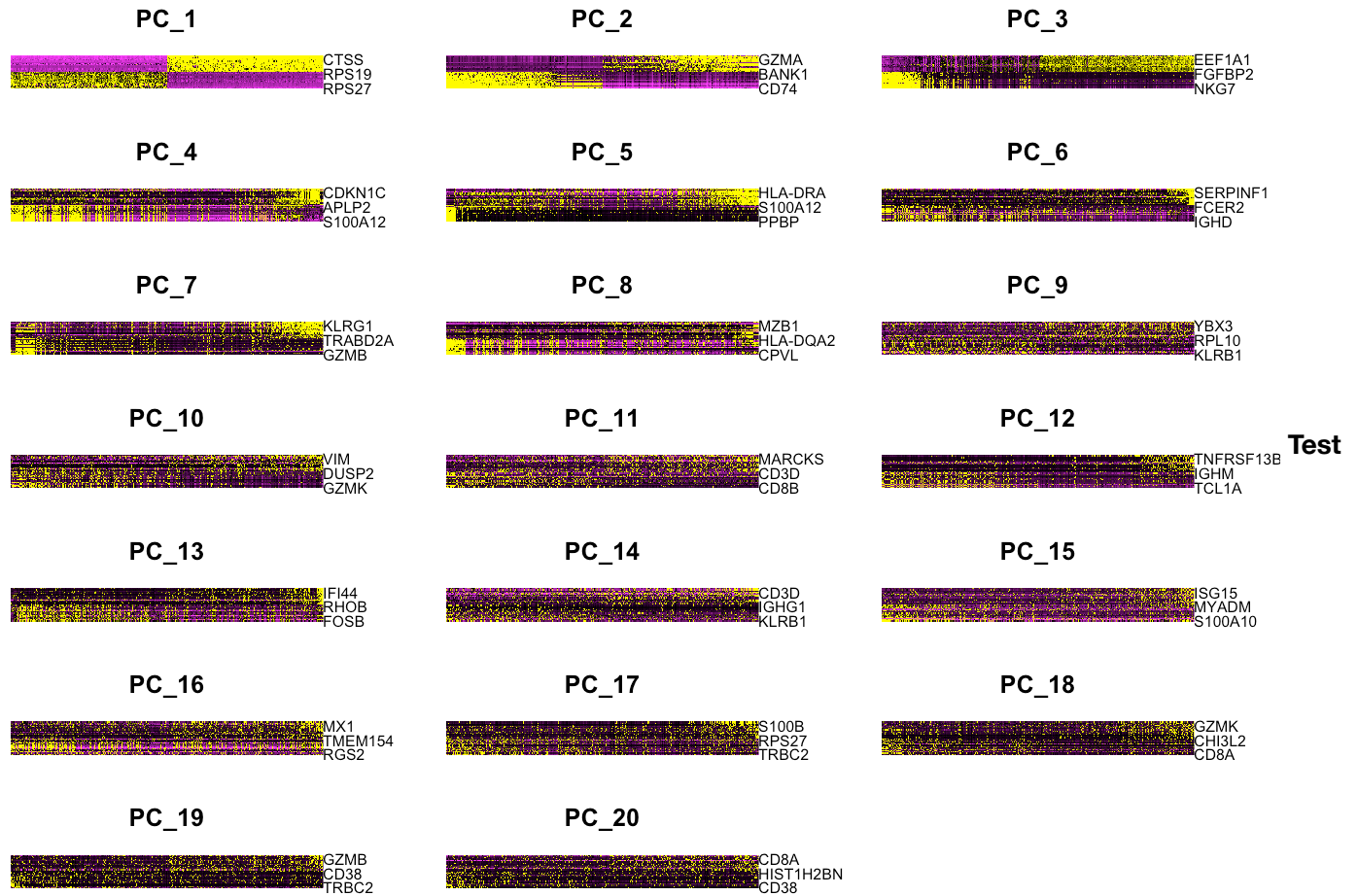


PC_15



#checking with 20 PCs

```
DimHeatmap(seurat_sct,
  dims = 1:20,
  cells = 500,
  balanced = TRUE)
```



different PC numbers

```
seurat_sct <- FindNeighbors(seurat_sct, dims = 1:10)
seurat_sct <- FindClusters(seurat_sct, resolution = 0.5)
```

```
## Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck
##
## Number of nodes: 1049
## Number of edges: 31308
##
## Running Louvain algorithm...
## Maximum modularity in 10 random starts: 0.8904
## Number of communities: 9
## Elapsed time: 0 seconds
```

```
n_clusters_10 <- length(unique(seurat_sct$seurat_clusters))

seurat_sct <- FindNeighbors(seurat_sct, dims = 1:15)
seurat_sct <- FindClusters(seurat_sct, resolution = 0.5)
```

```
## Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck
##
## Number of nodes: 1049
## Number of edges: 33136
##
## Running Louvain algorithm...
## Maximum modularity in 10 random starts: 0.8897
## Number of communities: 9
## Elapsed time: 0 seconds
```

```
n_clusters_15 <- length(unique(seurat_sct$seurat_clusters))

seurat_sct <- FindNeighbors(seurat_sct, dims = 1:20)
seurat_sct <- FindClusters(seurat_sct, resolution = 0.5)
```

```
## Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck
##
## Number of nodes: 1049
## Number of edges: 34535
##
## Running Louvain algorithm...
## Maximum modularity in 10 random starts: 0.8849
## Number of communities: 10
## Elapsed time: 0 seconds
```

```
n_clusters_20 <- length(unique(seurat_sct$seurat_clusters))

seurat_sct <- FindNeighbors(seurat_sct, dims = 1:25)
seurat_sct <- FindClusters(seurat_sct, resolution = 0.5)
```

```
## Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck
##
## Number of nodes: 1049
## Number of edges: 36778
##
## Running Louvain algorithm...
## Maximum modularity in 10 random starts: 0.8820
## Number of communities: 9
## Elapsed time: 0 seconds
```

```
n_clusters_25 <- length(unique(seurat_sct$seurat_clusters))
cat("Clusters with 10 PCs:", n_clusters_10, "\n")
```

```
## Clusters with 10 PCs: 9
```

```
cat("Clusters with 15 PCs:", n_clusters_15, "\n")
```

```
## Clusters with 15 PCs: 9
```

```
cat("Clusters with 20 PCs:", n_clusters_20, "\n")
```

```
## Clusters with 20 PCs: 10
```

```
cat("Clusters with 25 PCs:", n_clusters_25, "\n")
```

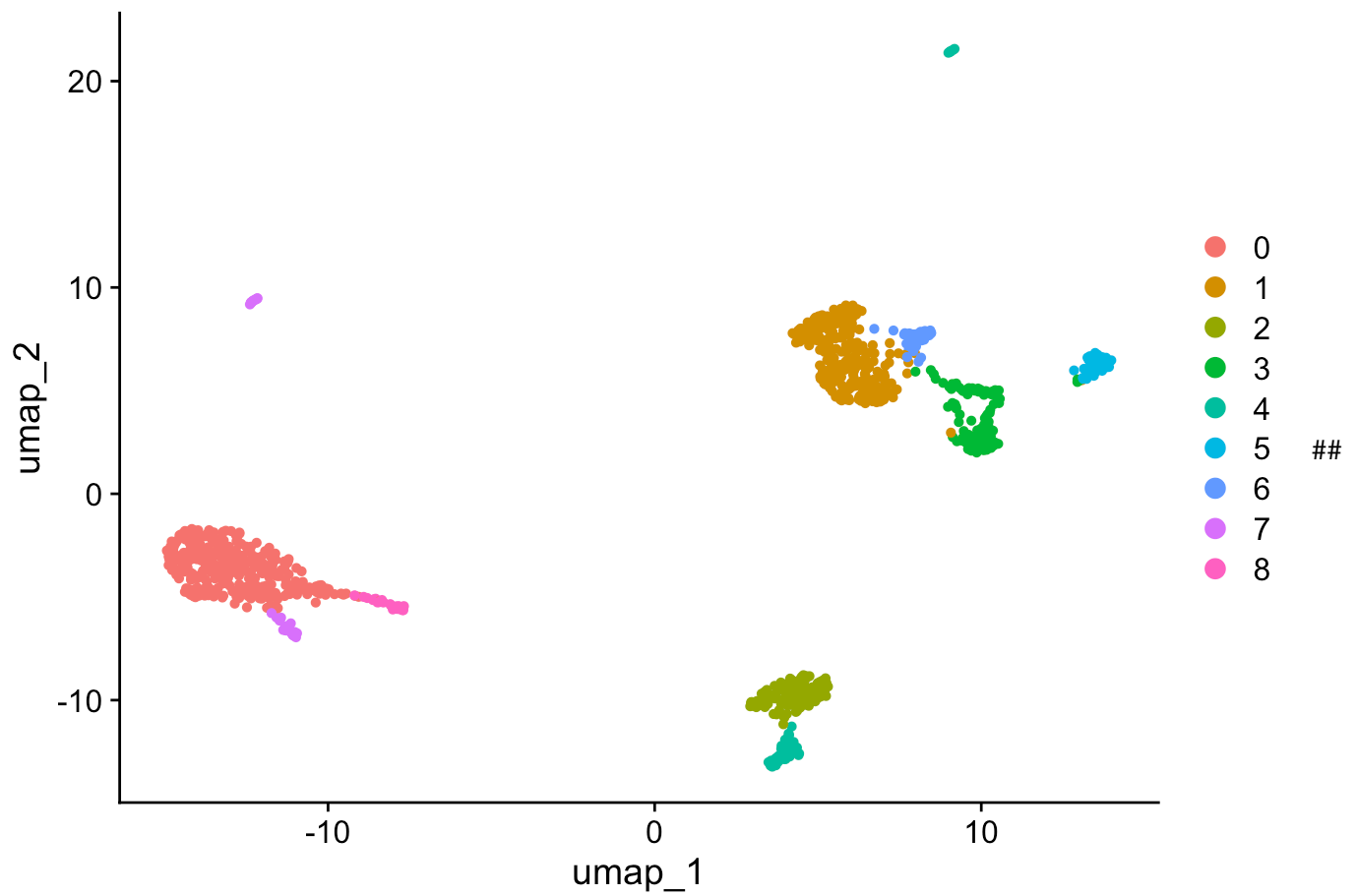
```
## Clusters with 25 PCs: 9
```

I assessed clustering stability across multiple PC choices. Clustering was largely stable between 10 and 25 PCs, yielding 9–10 clusters. The appearance of an additional cluster at 20 PCs suggested that PCs 16–20 captured additional biological variation. Combined with the elbow plot and DimHeatmap analysis showing biologically meaningful gene signatures up to PC20, I selected 20 PCs for downstream analyses.

Q2.7 UMAP VS tSNE

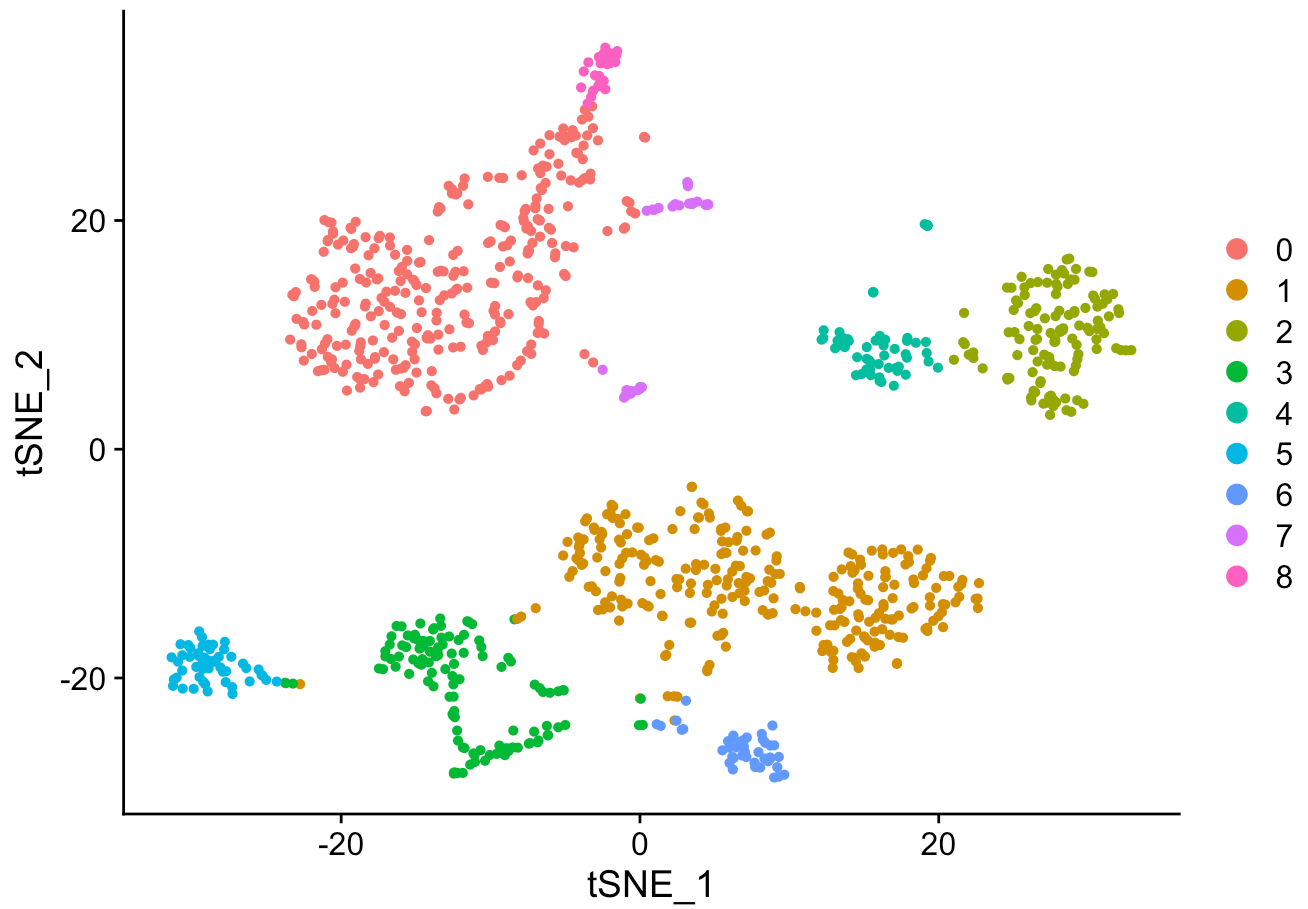
t-SNE focuses on preserving local relationships between cells, ensuring that similar cells remain close together, but distances between clusters are often not meaningful. UMAP also preserves local neighborhoods but attempts to retain more of the overall structure of the data by constructing and embedding a nearest-neighbor graph. This is why, UMAP is generally faster and better at visualizing both clusters and continuous biological transitions. ## Running UMAP

```
seurat_sct <- RunUMAP(seurat_sct, dims = 1:20) # 20 PCs used  
DimPlot(seurat_sct, reduction = "umap")
```

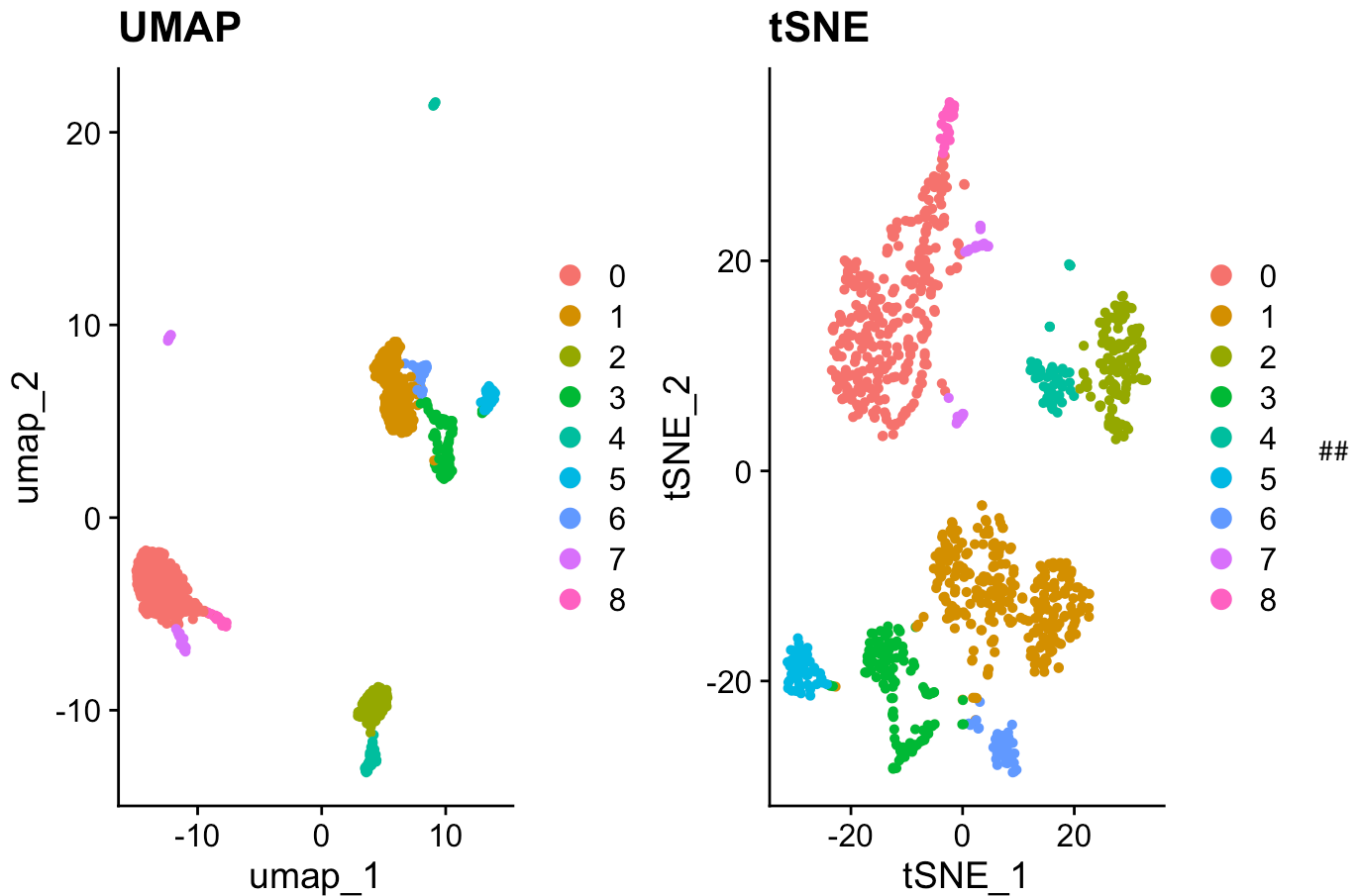


Running tSNE

```
seurat_sct <- RunTSNE(seurat_sct, reduction = "pca", dims = 1:20, perplexity = 20, seed.  
use = 1) # 20 PCs used  
DimPlot(seurat_sct, reduction = "tsne")
```



```
p1 <- DimPlot(seurat_sct, reduction = "umap") + ggtitle("UMAP")  
p2 <- DimPlot(seurat_sct, reduction = "tsne") + ggtitle("tSNE")  
p1 + p2
```



Result Interpretation

In the UMAP plot, clusters are arranged such that cells with similar gene expression profiles tend to be located closer together in the embedding. While the distances are not exact measures of biological similarity, nearby clusters often represent more closely related cell populations. For example, clusters 2 and 3 are positioned close to each other, suggesting they may share similar transcriptional characteristics. This is one of the advantages of UMAP, as it attempts to preserve both local neighborhoods and some of the broader structure of the dataset.

In contrast, t-SNE focuses primarily on preserving local relationships between cells. It is very effective at separating clusters, but the distances between clusters are generally not biologically interpretable. We can see clusters being pushed farther apart in the t-SNE plot due to the repulsive forces used to overcome the crowding problem. As a result, while t-SNE clearly highlights distinct cell populations, we cannot reliably infer how similar or dissimilar two clusters are based solely on the distance between them in the embedding.

Q2.8 — Clustering Resolution Strategy

Clustering resolution controls the granularity of cell groupings. Higher the resolution creates more clusters. We need to find the right spot where the resolution sits just right without over clustering or under clustering, with the clusters actually portraying biological diversity.

I have used the clustree analysis to see how the clusters are splitting :


```
library(clustree)
```

```
# Trying different resolutions
```

```
resolutions <- c(0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.8, 1.0)
```

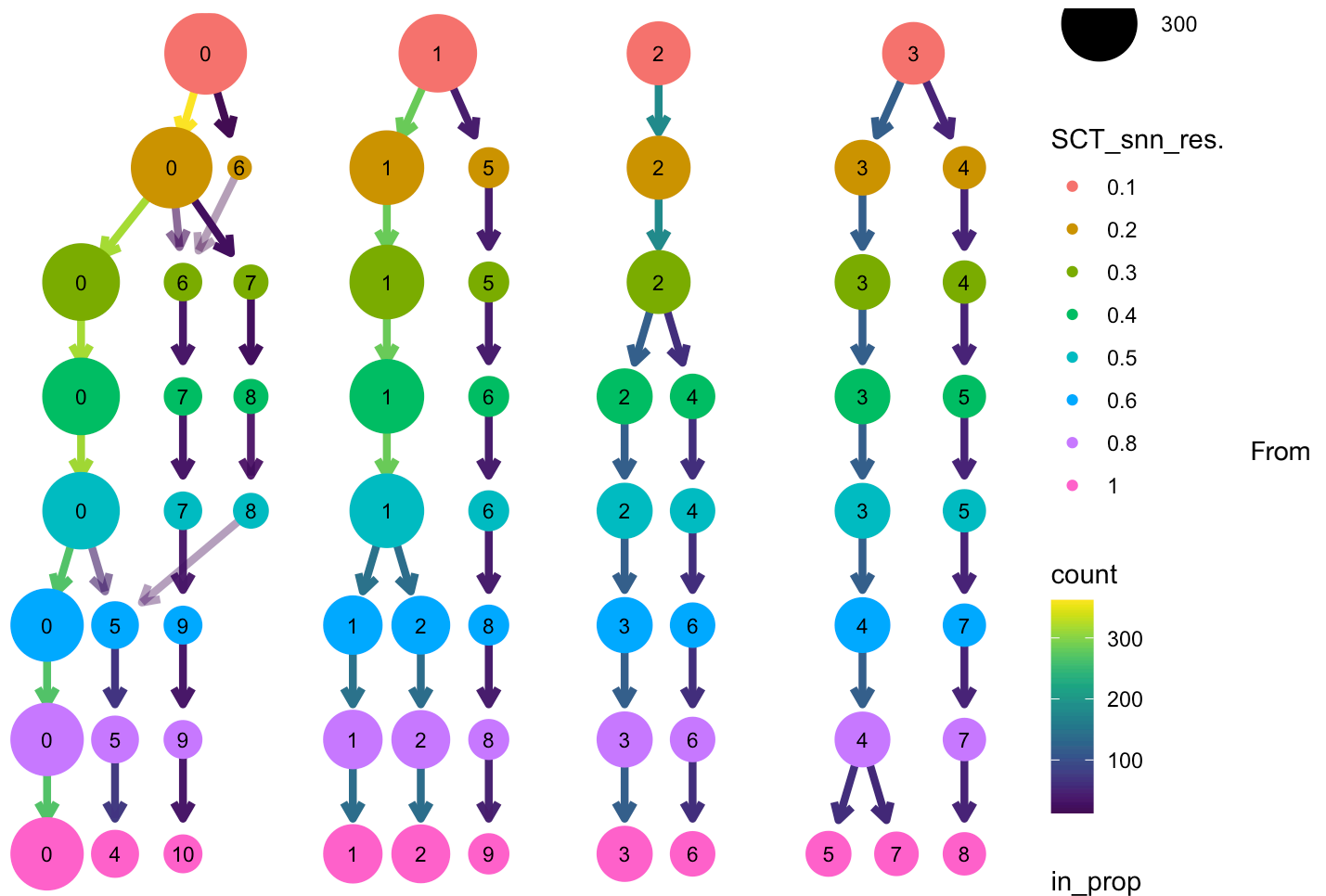
```
for(res in resolutions){
```

```
  seurat_sct <- FindClusters(seurat_sct,
                             resolution = res,
                             verbose = FALSE)
```

```
}
```

```
# shows how the clusters are split across different resolutions and we can choose the most stable one
```

```
clustree(seurat_sct, prefix = "SCT_snn_res.")
```



- **Resolution 0.1-0.4** — too coarse, shows distinct populations being merged incorrectly and lots of splitting occurs.
- **Resolution 0.4-0.5** — clusters show stable, thick straight arrows indicating robust biological groupings. Almost nothing is changing.
- **Resolution 0.5-0.6** — Here we see a new cluster, 1 splits to 1 and 2 and cluster 0 to 0.5 and 9. So, 0.6 is capturing additional heterogeneity.
- **Resolution 0.6-0.8** — The clusters identified at 0.6 persist. Very little changes.
- **Resolution 0.8-1.0** — One additional split occurs, this can be a rare population or overclustering.

Major cluster splits are occurring between resolutions 0.2 and 0.6, indicating the resolution of previously merged populations. Resolution 0.6 produced 10 clusters, and these clusters remained largely stable through resolution 0.8, suggesting that these identified communities are robust. Increasing the resolution to 1.0 resulted in an additional cluster. Therefore, I decided to check markers in this cluster to see if this is a rare population.

```
seurat_sct <- FindClusters(seurat_sct,
                           resolution = 1.0,
                           verbose = FALSE)

# Find markers for the new cluster 10
cluster10_markers <- FindMarkers(seurat_sct,
                                  ident.1 = 10,
                                  min.pct = 0.25)

# Show top 10 markers in cluster 10
head(cluster10_markers, 10)
```

##		p_val	avg_log2FC	pct.1	pct.2	p_val_adj
##	CLDN5	1.898820e-59	9.141961	0.286	0.001	2.614295e-55
##	CMTM5	2.852319e-56	8.663914	0.314	0.003	3.927073e-52
##	TMEM40	4.751397e-53	8.731028	0.257	0.001	6.541723e-49
##	GP9	4.137870e-49	8.026484	0.343	0.007	5.697020e-45
##	TREML1	5.741469e-49	7.225793	0.343	0.007	7.904855e-45
##	CTTN	2.434931e-43	7.629148	0.286	0.005	3.352413e-39
##	C19orf33	4.306080e-43	6.078951	0.286	0.005	5.928611e-39
##	TUBB1	6.046007e-42	7.780323	0.371	0.013	8.324142e-38
##	ITGB5	3.043415e-40	5.856559	0.286	0.006	4.190174e-36
##	GMPR	8.961877e-39	5.567052	0.314	0.009	1.233871e-34

Cluster 10 shows very strong, biologically relevant distinct markers including CLDN5, GP9, TREML1 and TUBB1 which are well established platelet and megakaryocyte markers with extremely high fold changes and highly significant adjusted p-values. These are most probably genes associated with platelet/megakaryocyte identity. Therefore, resolution 1.0 can be preferred when rare cell populations are of interest. Nevertheless, resolution 0.6 is retained for the primary downstream analyses because it provided a stable clustering solution while avoiding excessive partitioning of major cell populations.

Plot at 0.6 resolution : -

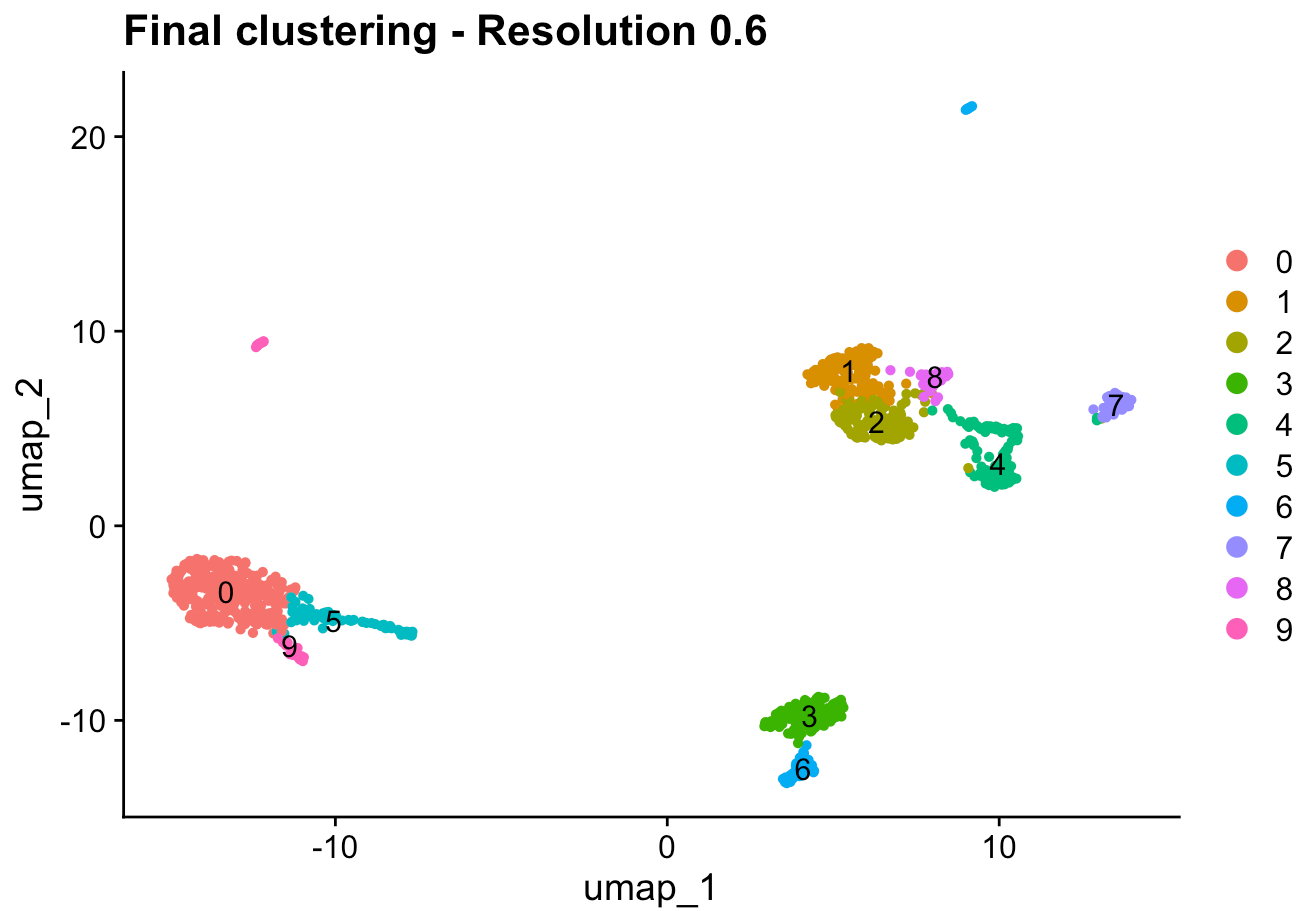
```
# Set final resolution to 0.6
seurat_sct <- FindClusters(seurat_sct,
                           resolution = 0.6,
                           verbose = FALSE)
cat("Final resolution: 0.6\n")
```

```
## Final resolution: 0.6
```

```
cat("Final clusters:",
    length(unique(seurat_sct$seurat_clusters)), "\n")
```

Final clusters: 10

```
DimPlot(seurat_sct,
        reduction = "umap",
        label = TRUE) +
  ggtitle("Final clustering - Resolution 0.6")
```



Q2.9 DEG Algorithms in Seurat

Seurat’s FindMarkers() and FindAllMarkers() functions support several differential expression testing methods through the test.use parameter. Each method is based on a different statistical approach and are suitable according to the characteristic of the dataset and biological question to be addressed.

Seurat’s FindMarkers() and FindAllMarkers() support multiple DE algorithms via the test.use parameter. Each uses a different statistical rationale suited to different biological contexts.

Algorithm	Rationale	Best Context	Limitations
wilcox (default)	Checks whether one group generally has higher expression than the other	Most scRNA-seq datasets, no distribution assumption	Ignores count distributions, inflated significance with large n

Algorithm	Rationale	Best Context	Limitations
bimod	Looks for genes that are ON in one group and OFF in another	Bimodally expressed genes	Assumes bimodal distribution
roc	Tests how well a gene can distinguish between two groups	Cell type marker identification	No p-values produced
t	Compares the average expression between groups	Approximately normal data	Assumes normality — violated in scRNA-seq
negbinom	Compares gene counts while accounting for the extra variability which is common in RNA-seq data	Raw count data	Slow, needs sufficient cells
poisson	Compares gene counts assuming variation comes only from counting noise	Low overdispersion UMI data	Mean=variance assumption often violated
LR	Tests whether gene expression can predict which group a cell belongs to	Controlling batch/sex confounders	Slow, requires many cells
MAST	Handles genes that are zero in many cells and expressed in only some cells	Zero-inflated scRNA-seq data	Slow, requires MAST package
DESeq2	Compares gene counts while carefully accounting for technical variation	Pseudobulk analysis	Too slow and not ideal when treating each cell as an individual sample

It is worth noting edgeR and limma-voom, although it is not directly built as test.use options in FindMarkers(), is widely used for differential expression in single-cell data through a pseudobulk approach aggregating counts per sample per cluster before applying bulk RNA-seq statistical frameworks. This approach is statistically more rigorous than single-cell level testing as it properly accounts for biological replicates and reduces false positives from inflated sample sizes. In my thesis work on Huntington's disease, I implemented pseudobulk DE using edgeR's GLM framework with explicit contrast definitions across experimental groups.

Q2.10 Seurat objects convertiong into different data formats

MEX format

```
library(DropletUtils)
# Extract counts correctly
counts_matrix <- GetAssayData(seurat_sct,
                             assay = "RNA",
                             layer = "counts")

#creating output directory
dir.create("MEX_output", showWarnings = FALSE)
# Write MEX format
write10xCounts(path = "MEX_output",
               x = counts_matrix,
               version = "3",
               overwrite = TRUE)

cat("MEX files saved!\n")
```

```
## MEX files saved!
```

```
list.files("MEX_output")
```

```
## [1] "barcodes.tsv.gz" "features.tsv.gz" "matrix.mtx.gz"
```

SingleCellExperiment

```
sce_final <- as.SingleCellExperiment(seurat_sct)
saveRDS(sce_final, "sce_final.rds")
cat("SCE saved!\n")
```

```
## SCE saved!
```

```
cat("Dimensions:", dim(sce_final), "\n")
```

```
## Dimensions: 13768 1049
```

Cloupe file

```
library(loupeR)
create_loupe_from_seurat(seurat_sct,
                         output_name = "demo_loupe",
                         force = TRUE)
```

Section 3 - BONUS

Q 3.1 Cell Ranger VS DropletUtils VS Cellbender

Cell Ranger, DropletUtils, and CellBender represent three different generations of software developed to tackle the issue of filetering droplets which has real cells from droplets that are just empty.

Cell ranger Rationale - Quick, standardized default calling based on total UMI based knee point detection algorithm. Main Advantage - Turnkey execution; completely integrated into the 10x Genomics ecosystem. Limitation - Often misses small, low-RNA cells or includes empty droplets. **DropletUtils** Rationale - Empty droplets have a distinct ambient sequence profile, real cells deviate from it. Main Advantage - Exceptional at identifying tiny or quiet cells without counting empty soup. Limitation - Does not physically clean ambient RNA out of count matrix. **Cellbender** Rationale - Every droplet is a mixture of a cell profile and a noise profile, model both simultaneously (Deep learning model). Main Advantage - Physically removes background soup counts and fixes cell calling at the same time. Limitation - Extremely slow; requires a GPU and significant computational power.

Q3.2 — Cell Cycle Assessment

Rationales

Seurat::CellCycleScoring(): Uses predefined data of S phase and G2M phase marker genes. For each cell it calculates two scores — an S score and a G2M score — based on the average expression of phase-specific marker genes. Cells are assigned to the phase with the highest score, or G1 if both scores are low. This is fast and straightforward but depends entirely on the quality of the predefined gene lists. It is almost like a teacher grading student based on fixed answers.

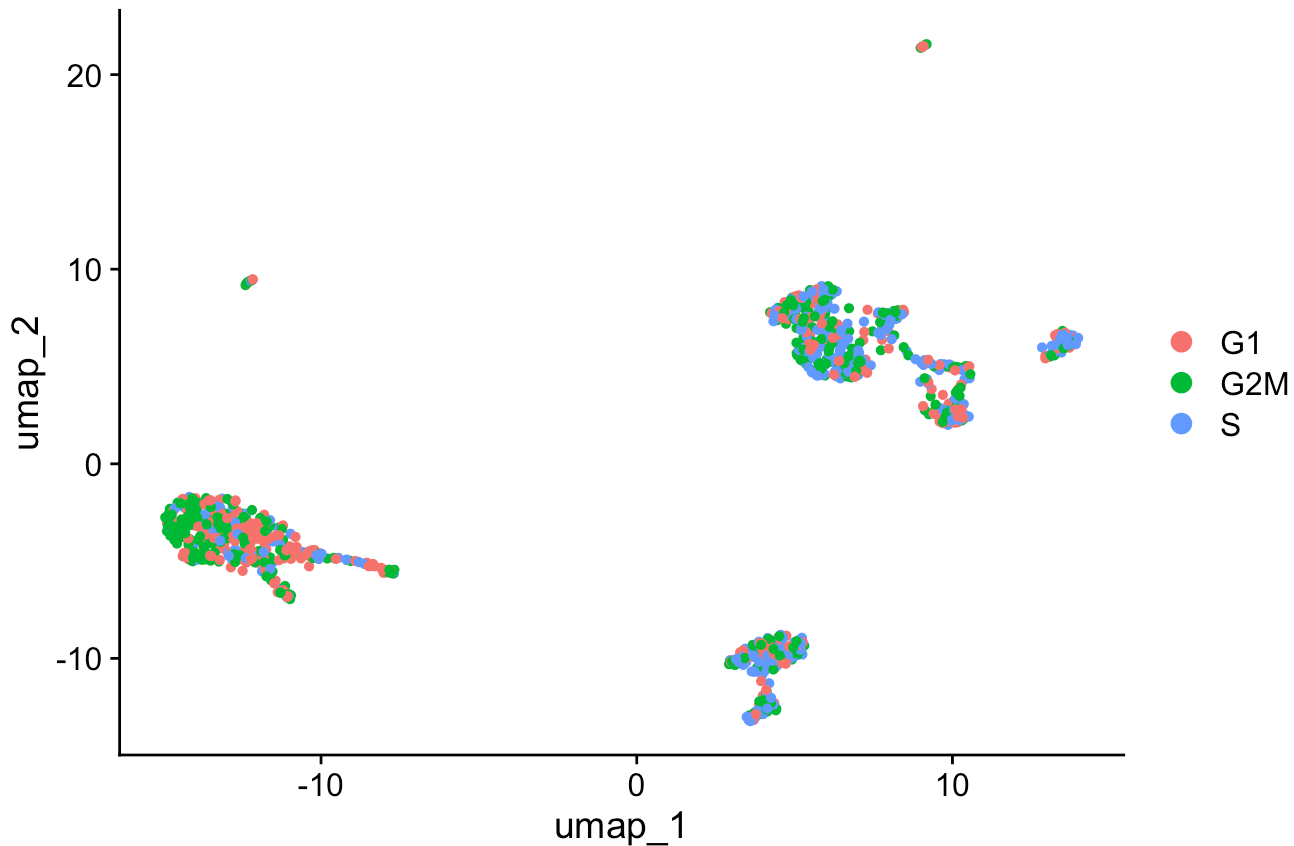
scran::cyclone(): Instead of checking if Gene A is “highly expressed,” cyclone asks: “Is Gene A expressed more than Gene B inside this specific cell?” Uses a pair-based approach which is trained on reference datasets. This is more statistically rigorous and works better for non-human species or datasets where predefined marker lists may not be optimal.

```
seurat_sct <- CellCycleScoring(
  object = seurat_sct,
  s.features = cc.genes$s.genes,
  g2m.features = cc.genes$g2m.genes,
  set.ident = FALSE
)
table(seurat_sct$Phase)
```

```
##
##  G1 G2M  S
## 303 408 338
```

```
DimPlot(seurat_sct,
  reduction = "umap",
  group.by = "Phase") +
  ggtitle("Cell Cycle Phase Distribution")
```

Cell Cycle Phase Distribution



Interpretation The dataset shows: - G1: 303 cells (29%) - G2M: 408 cells (39%) - S: 338 cells (32%) From this we can see high proportion of G2M and S phase cells which suggests an actively proliferating population. Importantly, the UMAP shows cell cycle phases are well mixed across all clusters no cluster is dominated by one single phase. This indicates cell cycle is not the primary driver of clustering in this dataset and regression of cell cycle effects may not be strictly necessary.

Q 3.3 - — False S/G2M in Post-Mitotic Tissue

The core issue is that Seurat's cell cycle tool is too literal, it uses a rigid checklist. It looks at a cell, sees that DNA repair genes or plasticity genes are highly turned on, it considers that cell is duplicating its DNA to divide. But this way it completely misses the biological context. Neurons are permanent, post-mitotic cells and they never divide. They are just using those specific 'cell-cycle' type genes to fix DNA damage from oxidative stress or to remodel their synapses. Because Seurat only checks if the volume of these genes is turned up high, it gets tricked by the busy baseline activity of a normal neuron. cyclone on the other hand fixes this because it doesn't care about absolute volume. It looks at the internal ratio or hierarchy of the genes (if geneA > gene B). Even if a neuron turns up a few specific repair genes, the overall ranking of its transcript profile still looks like a resting cell, so cyclone correctly labels it as non-dividing. This working difference makes seurat falsely report higher S/G2M cells.

Q 3.4 — TransferData vs SingleR

Seurat::TransferData(): Finds mutual nearest neighbour "anchors" between reference and query in PCA space (both the reference data set and query data set supposed to mutually agree to the output), then transfers labels through those anchors. Confidence scores are included.

SingleR(): Calculates Spearman correlation between each query cell and reference cell type profiles. Assigns cell type with highest correlation and the Fine-tuning step re-runs on top candidates to resolve ambiguous assignments.

Key difference: TransferData requires finding shared structure between datasets through anchors — can struggle when datasets are very dissimilar. SingleR's correlation approach is more robust to dataset differences as no anchor identification is needed.

Q 3.5 - Handling Batch Effects in Cell Type Annotation

If strong batch effects exist, SingleR would be the preferred choice over TransferData. This is because TransferData's annotation depends on finding anchors in shared PCA space, if strong batch effects reduces anchor quality and number, the label transfer would be poor. SingleR's Spearman correlation approach doesn't require dataset alignment, making it more robust to systematic differences between datasets.

Recent Methods : these new tools use neural networks to clean the batch effect while annotating.

scArches (Single-cell Architecture Surgery) : Developed to solve the exact problem of strong batch effects, scArches uses a deep-learning architecture called a Variational Autoencoder (VAE).

CellTypist : Instead of relying on a single reference dataset, CellTypist models are trained across dozens of different tissue cohorts simultaneously. Because the algorithm has already seen a T-cell through the lens of twenty different labs, sequencing technologies, and batch conditions, its classification boundaries are incredibly resilient.

Q 3.6 - aggr vs RPCA vs Harmony

1. Cell Ranger aggr

Rationale : Cell Ranger aggr handles the simplest batch effect of all - sequencing depth. If Batch 1 was sequenced deeply for e.g., 100,000 reads per cell and Batch 2 was sequenced shallowly 20,000 reads per cell, they will cluster separately just because of machine numbers. aggr fixes this by looking at the shallowest sample, and then randomly throwing away reads from the deeper samples until all libraries are at the exact same average sequencing depth. Limitations : It only normalizes sequencing depth. It cannot fix true biological or chemical batch effects. Data Waste as it literally discards high-quality data just to make the samples look technically equal.

2. **Seurat RPCA (Robust Principal Component Analysis)** Rationale : Instead of throwing all genes from all samples into a huge blender, it identifies the core, stable biological genes or mutual nearest neighbours in shared PCA space (anchors). It projects Dataset A into Dataset B's mathematical space and vice-versa using restricted, robust dimensions. It then finds where the cells overlap across datasets and draws structural lines (anchors) to knit them together. Limitations : Because it only aligns data based on highly stable, shared features, it is a bit rigid. If the datasets have highly subtle or unique cell states, RPCA might completely miss them or fail to bring the clusters completely together. Also, if Dataset A is brain tissue and Dataset B is liver tissue, RPCA will break down because there is no true shared global structure to anchor.

3. **Harmony** Rationale : It doesn't look for rigid single-cell agreements. Instead, it takes all the data, combines it into a shared PCA space, and looks at the clusters. If it sees a cluster that is made up only of Batch 1 cells and another of Batch 2 cells right next to it, it asks if they are actually different cell types, or is this just batch noise? Then it calculates a correction vector for each individual cell and gently nudges the cell clusters together until they blend into a diverse, multi-batch mixture. Limitations : Harmony is highly aggressive. If your Batch 1 is healthy tissue and Batch 2 is tumor tissue, Harmony might assume the

massive biological difference is just a technical batch effect and forcefully blend them together, destroying your true experimental biology. And Because it continuously adjusts cell coordinates iteratively in an abstract mathematical space, it can be harder to untangle exactly how much of raw data was modified.

Q3.7 — Data Integration: Merge, RPCA, and Harmony

```
library(harmony)
seurat_integrate <- readRDS("/Users/ask/Downloads/data_scRNA_2/to_integrate.rds")
options(future.globals.maxSize = 2000 * 1024^2)

# Name objects and add metadata
names(seurat_integrate) <- c("AD1", "AD2", "Control1", "Control2")
for(sample in names(seurat_integrate)){
  seurat_integrate[[sample]]$sample <- sample
  seurat_integrate[[sample]]$condition <- ifelse(
    grepl("AD", sample), "AD", "Control")
}

# Method 1 – Simple merge
seurat_merged <- merge(
  seurat_integrate[["AD1"]],
  y = list(seurat_integrate[["AD2"]],
           seurat_integrate[["Control1"]],
           seurat_integrate[["Control2"]]),
  add.cell.ids = c("AD1", "AD2", "Control1", "Control2")
)
seurat_merged <- NormalizeData(seurat_merged) %>%
  FindVariableFeatures() %>%
  ScaleData() %>%
  RunPCA(verbose = FALSE) %>%
  FindNeighbors(dims = 1:20) %>%
  FindClusters(resolution = 0.5, verbose = FALSE) %>%
  RunUMAP(dims = 1:20)

# Method 2 – RPCA
seurat_list <- lapply(seurat_integrate, function(x) {
  x <- NormalizeData(x, verbose = FALSE)
  x <- FindVariableFeatures(x, verbose = FALSE)
})
anchors_rpca <- FindIntegrationAnchors(
  object.list = seurat_list,
  reduction = "rpca", dims = 1:20, verbose = FALSE)
cat("RPCA anchors found:", nrow(anchors_rpca@anchors), "\n")
```

```
## RPCA anchors found: 9978
```

```

seurat_rpca <- IntegrateData(anchorset = anchors_rpca,
                             dims = 1:20, verbose = FALSE)
DefaultAssay(seurat_rpca) <- "integrated"
seurat_rpca <- ScaleData(seurat_rpca, verbose = FALSE) %>%
  RunPCA(verbose = FALSE) %>%
  FindNeighbors(dims = 1:20) %>%
  FindClusters(resolution = 0.5, verbose = FALSE) %>%
  RunUMAP(dims = 1:20)

# Method 3 – Harmony
seurat_harmony <- seurat_merged
seurat_harmony <- RunHarmony(seurat_harmony,
                             group.by.vars = "sample",
                             verbose = FALSE)
seurat_harmony <- FindNeighbors(seurat_harmony,
                                reduction = "harmony", dims = 1:20) %>%
  FindClusters(resolution = 0.5, verbose = FALSE) %>%
  RunUMAP(reduction = "harmony", dims = 1:20)

```

Q3.8 — Integration Performance Evaluation

For the merged dataset, cells show partial grouping by sample, which indicates that there is still residual batch effects. Which is expected as merged doesn't necessarily correct anything mathematically. In RPCA we can see improved mixing of cells across samples while maintaining clear separation of major cell types, suggesting better correction of technical variation. We can see much more blended clusters irrespective of sample and the condition plot looks distinct from one another. Harmony actually produced the strongest sample mixing, with cells from all samples overlapping extensively while preserving major cell type structure. Overall, both RPCA and Harmony substantially reduced batch effects compared to simple merging. But the concern with Harmony is that it may have aggressively smushed in the cells together as we can see in the condition plot this can remove the biological variation, it could have taken away some real AD biology. So even though Harmony showed the best batch correction, RPCA provided a good balance between batch removal and preservation of biological variation.

Q3.9 Alternative methods to RPCA if there are less anchors in dataset

RPCA is sensitive to anchor quality because it relies on mutual nearest neighbours in shared PCA space to correct batch effects. When datasets are highly dissimilar like different cell type compositions, sequencing platforms, or tissue sources they create fewer high-confidence anchors, leading to incomplete correction. If insufficient anchors are identified by RPCA, alternative methods such as Harmony can be used. Harmony is particularly useful because it corrects batch effects in low-dimensional PCA space and does not rely heavily on anchor identification.

3.10 - DEGs Associated with AD per Cell Type

```
library(openxlsx)
seurat_rpca <- JoinLayers(seurat_rpca, assay = "RNA") # Join layers first (required for Seurat v5)

# Set identity to cell type
Idents(seurat_rpca) <- seurat_rpca$paper_annot
DefaultAssay(seurat_rpca) <- "RNA"

# Get cell types
cell_types <- unique(seurat_rpca$paper_annot)

# Run DEG analysis per cell type
deg_results <- list()

for(ct in cell_types){
  n_AD <- sum(seurat_rpca$condition[seurat_rpca$paper_annot == ct] == "AD")
  n_Control <- sum(seurat_rpca$condition[seurat_rpca$paper_annot == ct] == "Control")

  if(n_AD >= 10 & n_Control >= 10){
    tryCatch({
      markers <- FindMarkers(
        seurat_rpca,
        ident.1 = "AD",
        ident.2 = "Control",
        group.by = "condition",
        subset.ident = ct,
        min.pct = 0.1,
        logfc.threshold = 0.25,
        verbose = FALSE
      )
      markers$gene <- rownames(markers)
      markers$cell_type <- ct
      deg_results[[ct]] <- markers
      cat(ct, "- DEGs found:", nrow(markers), "\n")
    }, error = function(e){
      cat("Skipping", ct, ":", e$message, "\n")
    })
  }
}
```

```
## Excitatory Neurons – DEGs found: 7618
## Inhibitory Neurons – DEGs found: 8851
## Oligodendrocytes – DEGs found: 5018
## Astrocytes – DEGs found: 7088
## OPCs – DEGs found: 7168
## Microglia – DEGs found: 5209
## Endothelial – DEGs found: 6843
## Pericytes – DEGs found: 6648
```

```
cat("\nTotal cell types processed:", length(deg_results), "\n")
```

```
##
## Total cell types processed: 8
```

```
summary_df <- data.frame(
  Cell_Type = names(deg_results),
  Total_DEGs = sapply(deg_results, nrow),
  Up_in_AD = sapply(deg_results, function(x) sum(x$avg_log2FC > 0)),
  Down_in_AD = sapply(deg_results, function(x) sum(x$avg_log2FC < 0))
)

print(summary_df)
```

##	Cell_Type	Total_DEGs	Up_in_AD	Down_in_AD
## Excitatory Neurons	Excitatory Neurons	7618	4253	3365
## Inhibitory Neurons	Inhibitory Neurons	8851	4381	4470
## Oligodendrocytes	Oligodendrocytes	5018	2974	2044
## Astrocytes	Astrocytes	7088	4014	3074
## OPCs	OPCs	7168	4214	2954
## Microglia	Microglia	5209	2571	2638
## Endothelial	Endothelial	6843	3431	3412
## Pericytes	Pericytes	6648	3332	3316

```
wb <- createWorkbook()

for(ct in names(deg_results)){
  sheet_name <- substr(gsub(" ", "_", ct), 1, 31)
  addWorksheet(wb, sheet_name)
  deg_sorted <- deg_results[[ct]][
    order(deg_results[[ct]]$p_val_adj), ]
  writeData(wb, sheet_name, deg_sorted)
}

saveWorkbook(wb, "AD_DEGs_per_celltype.xlsx", overwrite = TRUE)
cat("Saved: AD_DEGs_per_celltype.xlsx\n")
```

```
## Saved: AD_DEGs_per_celltype.xlsx
```

```
cat("Sheets:", paste(names(deg_results), collapse = ", "), "\n")
```

```
## Sheets: Excitatory Neurons, Inhibitory Neurons, Oligodendrocytes, Astrocytes, OPCs, Microglia, Endothelial, Pericytes
```